Collecting and Analyzing Social Media Data

Francesca Giannetti

Table of contents

1 About 2

2 Prerequisites 2

3 Characteristics of Social Media 2

4 About Bluesky 3

5 About Bluesky 2 3

6 About Bluesky 3 3

7 About Bluesky: Technical 3

8 Get an App Password 3

9 Create Session 4

10 A few examples of data collection 4

11 Query specific handles 4

12 Fetch posts from custom feeds 5

13 Fetch user lists or starter packs 5

14 Query by keywords 7

15 Activity 1 7

16 Manipulating and Visualizing Social Media Data 7

17 Hashtag Analysis 7

18 Image URLs 9

19 Time Series Analysis 10

20 Activity 2 12

21 Export results to CSV 12

22 A note on Gen AI 13

23 Thanks 13

1 About

Social media data provide researchers with a real-time view into a wide variety of social and cultural topics. In this workshop, we will use handful of Bluesky endpoints to collect data. We will then explore the resulting dataset using the tidyverse collection of packages in R.

This workshop will address the kinds of information we can glean from social media data as well as several important factors and limitations to consider when doing social media research.

2 Prerequisites

For this workshop, you'll need a Bluesky account and a Posit Cloud account. Create and/or log into them now. I recommend using Google/Scarletmail credentials to set up the free Posit Cloud account.

- Bluesky
- Posit Cloud

3 Characteristics of Social Media

It is often held that social media data provide critical and unique insights into human behavior. And social media platforms offer a relatively economical, quick, and targeted way of collecting data, especially as compared to some other sources, e.g. polls, surveys, demographic and economic data. And yet each social media platform presents with structural biases that are important to consider.[1]

[1] Tufekci provides a strong overview of the methodological considerations of using social media data for research in "Big Questions for Social Media Big Data: Representativeness, Validity and Other Methodological Pitfalls".

4 About Bluesky

- According to Pew, just 4% of U.S. adults use it
- They skew young, white/Hispanic, democratic, well educated, suburban
- In 2025, it was the platform with the second fewest users (Truth Social has fewest)
- 4.25 million active users (last 30 days)

5 About Bluesky 2

- Launched in 2019 within Twitter; became independent in 2021; invitation only service in Feb 2023; public registrations in Feb 2024
- Academics flocked here after Twitter/X became too toxic
- Becoming a favored data source for computational social science

6 About Bluesky 3

These limitations are important to keep in mind as you frame your research question. For instance, we can't make generalizations about the population as a whole based on Bluesky data. We can however measure short-term effects, and look for insights relating to the variables we've identified as being implicated in our research question. As a purely practical tip, it may make sense to investigate larger topics that generate lots of activity, rather than smaller topics where the resulting dataset may be a less faithful representation of the subject or phenomenon we want to investigate.

7 About Bluesky: Technical

- Similar interface and interaction model as Twitter but different in that it uses Authenticated Transfer (AT) Protocol
- Authentication to their API requires an account-based app password, not an API key
- Depending on what your script needs to accomplish, route to the appropriate endpoint
- We will use an unauthenticated connection to query public data

8 Get an App Password

- In Bluesky, navigate to Settings > Privacy and Security > App Passwords
- Press Add App Password

- Enter your account name and app password and paste them into the indicated parts of the code block below

```
# get credentials

bsky_identifier <- "YOUR BLUESKY HANDLE"
bsky_app_password <- "YOUR APP PASSWORD"
```

9 Create Session

Here we initialize an HTTP request object in `httr2` targeting Bluesky's `createSession` endpoint. We also secure a JSON Web Token (JWT), which acts as our digital key—passing it in subsequent API requests proves to Bluesky that we are authenticated without needing to send our password again.

```
# create a session
auth_resp <- request("https://bsky.social/xrpc/com.atproto.server.createSession") |>
  req_method("POST") |>
  req_body_json(list(
    identifier = bsky_identifier, # Your handle
    password   = bsky_app_password     # Generated in Bluesky settings
  )) |>
  req_perform() |>
  resp_body_json()

# Get the Access JSON Web Token (JWT)
access_token <- auth_resp$accessJwt
```

10 A few examples of data collection

Here are a handful of examples of queries you can perform using various API endpoints.

11 Query specific handles

```r
# Search posts by a given author
elsayed_req <- request("https://public.api.bsky.app/xrpc/app.bsky.feed.getAuthorFeed") |>
  req_url_query(
    actor = "abdulelsayed.bsky.social", # Example handle: Abdul El-Sayed
    limit = 10
  )

elsayed_resp <- elsayed_req |> req_perform() |> resp_body_json() # parse JSON response

# Extract post texts
sapply(elsayed_resp$feed, function(item) item$post$record$text)
```

12 Fetch posts from custom feeds

```r
# Query a custom feed generator URI
# Example feed URI format: at://<CREATOR_DID>/app.bsky.feed.generator/<RECORD_KEY>
# Extract creator handle and record key from the feed's share link
academicsky_req <- request("https://public.api.bsky.app/xrpc/app.bsky.feed.getFeed") |>
  req_url_query(
    feed = "at://did:plc:4jb3re5tvklsvhuc3lkerj5q/app.bsky.feed.generator/aaak6p5s2f3cu", # I
    limit = 20
  )

academicsky_resp <- academicsky_req |> req_perform() |> resp_body_json()

# Extract and view post texts
academicsky_posts <- academicsky_resp$feed
sapply(academicsky_posts, function(item) item$post$record$text)
```

13 Fetch user lists or starter packs

We can get these by using the `app.bsky.graph.getList` and `app.bsky.graph.getStarterPack` endpoints.

```r
# Fetch members of a curated list
# For example, the dhmakes list published at https://bsky.app/starter-pack/literaturegeek.bsl

# First: resolve creator handle DID
```

```r
resolve_req <- request("https://public.api.bsky.app/xrpc/com.atproto.identity.resolveHandle")
  req_url_query(handle = "literaturegeek.bsky.social")

creator_did <- (resolve_req |> req_perform() |> resp_body_json())$did

# Second: construct starter pack AT-URI
# Format: at://<DID>/app.bsky.graph.starterpack/<RKEY>
starter_pack_rkey <- "3kxucwdkvd222"
starter_pack_uri  <- paste0("at://", creator_did, "/app.bsky.graph.starterpack/", starter_pa

# Third: Fetch starter pack metadata
dhmakes_req <- request("https://public.api.bsky.app/xrpc/app.bsky.graph.getStarterPack") |>
  req_url_query(
    starterPack = starter_pack_uri
  )
dhmakes_resp <- dhmakes_req |> req_perform() |> resp_body_json()

# Fourth: Extract the list URI backing the starter pack
list_uri <- dhmakes_resp$starterPack$list$uri

# Fifth: Fetch member profiles from the backing list
list_req <- request("https://public.api.bsky.app/xrpc/app.bsky.graph.getList") |>
  req_url_query(
    list  = list_uri,
    limit = 100
  )

list_resp <- list_req |> req_perform() |> resp_body_json()

# Sixth: Extract member handles into a vector
dhmakes_handles <- purrr::map_chr(list_resp$items, ~ .x$subject$handle)

# Seventh: View list of handles
head(dhmakes_handles)
```

```
[1] "ianthis.bsky.social"           "sophie-rose-reads.bsky.social"
[3] "conniebdowell.bsky.social"     "spottyboypress.bsky.social"
[5] "markalanmattes.bsky.social"    "slavagreenberg.bsky.social"
```

14 Query by keywords

```
letterpress_req <- request("https://bsky.social/xrpc/app.bsky.feed.searchPosts") %>%
  req_url_query(q = 'dhmakes letterpress',
                limit = 50) %>%
  req_auth_bearer_token(access_token) %>% # passes our bearer token
  req_perform()

letterpress_resp <- resp_body_json(letterpress_req)

sapply(letterpress_resp$posts, function(p) p$record$text)
```

15 Activity 1

Reflect on what interests you about social media data. Consider:

- What or who do you care about?
- How will you get at this data?
- Which endpoint will you need?
- Try your hand at crafting your query (Gen AI helps, if you're not averse)

16 Manipulating and Visualizing Social Media Data

Here are a handful of ways that you may wish to analyze the posts or handles you collect.

17 Hashtag Analysis

```
# install.packages("tidytext")
library(tidytext)

# construct keyword query (AcademicSky again)
academicsky_req <- request("https://bsky.social/xrpc/app.bsky.feed.searchPosts") |>
  req_auth_bearer_token(access_token) |>  # passes our bearer token
  req_url_query(q = 'academicsky',
                limit = 50) |>
  req_perform()
```

```r
academicsky_resp <- resp_body_json(academicsky_req)

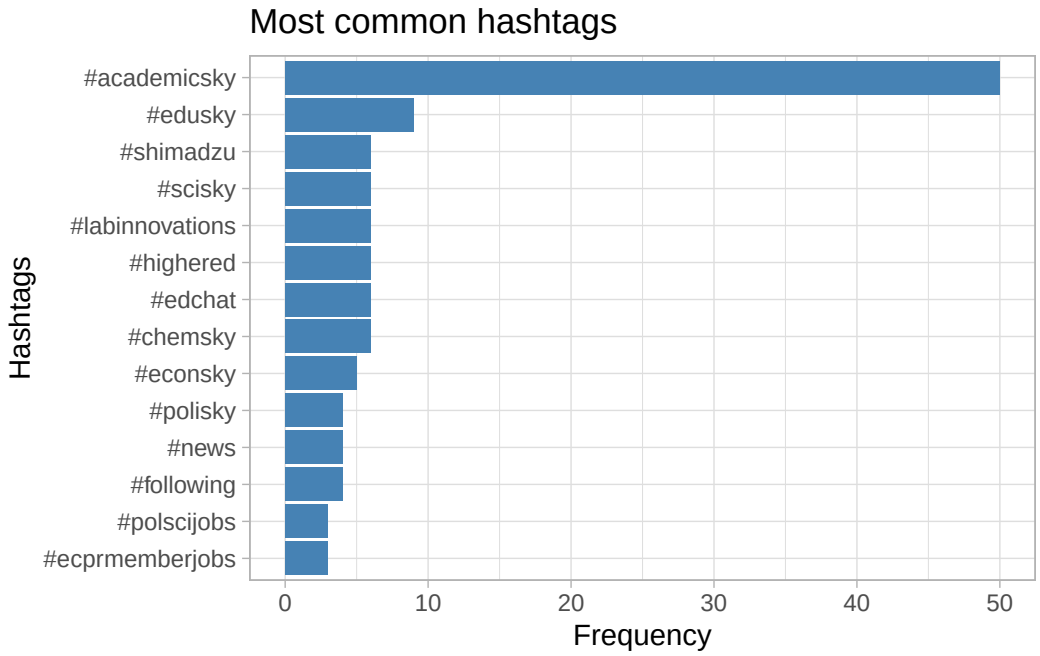
# reformat our posts as a tidy dataframe
academic_posts_df <- purrr::map_dfr(academicsky_resp$posts, \(post) {
  tibble(
    uri        = post$uri %||% NA_character_,
    author     = post$author$handle %||% NA_character_,
    text       = post$record$text %||% NA_character_,
    created_at = post$record$createdAt %||% NA_character_,
    replies    = post$replyCount %||% 0,
    likes      = post$likeCount %||% 0,
    reposts    = post$repostCount %||% 0
    )
})

reg_words <- "([^A-Za-z_\\d#@']|'(?![A-Za-z_\\d#@]))" # to tokenize text of posts
url_pattern <- "https?://\\S+|www\\.\\S+|&[a-z]+;|\\b(RT|MT)\\b|https?$"

tidy_academic_posts <- academic_posts_df |>
  mutate(text = str_replace_all(text, url_pattern, "")) |>
  # Extra step: Clean up extra white space left behind after removals
  mutate(text = str_squish(text)) |>
  unnest_tokens(word, text, token = "regex", pattern = reg_words) |>
  filter(!word %in% stop_words$word,
         str_detect(word, "[a-z]"))

tidy_academic_posts |>
  filter(str_detect(word, "^#")) |>  #just capture hashtags, comment out if unwanted
  count(word, sort=TRUE) |>
  filter(n > 2) |>
  mutate(word = reorder(word, n)) |>
  ggplot(aes(x=word, y=n)) +
  geom_col(fill = "steelblue") +
  labs(x = 'Hashtags', y = "Frequency", sep = "", title="Most common hashtags") +
  theme_light() +
  coord_flip()
```

```
Ignoring unknown labels:
* sep : ""
```



18 Image URLs

```
letterpress_req <- request("https://bsky.social/xrpc/app.bsky.feed.searchPosts") |>
  req_auth_bearer_token(access_token) |>
  req_url_query(
    q = "dhmakes letterpress",
    limit = 50
  ) |>
  req_perform()

letterpress_resp <- letterpress_req |> resp_body_json()

# helper function to extract image URLs
get_image_urls <- function(post) {
  embed <- post$embed
  if (is.null(embed)) return(NA_character_)

  # Case 1: Standard Image Post
  if (embed$`$type` == "app.bsky.embed.images#view") {
    return(map_chr(embed$images, ~ .x$fullsize) %>% paste(collapse = "; "))
  }
```

```
  # Case 2: Image Post with a Quote (Record with Media)
  if (embed$`$type` == "app.bsky.embed.recordWithMedia#view") {
    return(map_chr(embed$media$images, ~ .x$fullsize) %>% paste(collapse = "; "))
  }

  return(NA_character_)
}

# Apply this in your dataframe
clean_posts_with_media <- map(letterpress_resp$posts, \(post) {
  tibble(
    author     = post$author$handle,
    text       = post$record$text,
    image_urls = get_image_urls(post), # Custom extraction
    has_image  = !is.na(image_urls)
  )
}) %>% list_rbind()

# Filter to only show posts that actually have images
posts_with_images <- filter(clean_posts_with_media, has_image == TRUE)

# Inspect first image url

posts_with_images$image_urls[1]
```

```
[1] "https://cdn.bsky.app/img/feed_fullsize/plain/did:plc:quschouau47zm4f3ntsw7j5y/bafkreic65
```

19 Time Series Analysis

```
#install.packages("scales")
library(scales)
```

```
Attaching package: 'scales'

The following object is masked from 'package:purrr':

    discard
```

```
The following object is masked from 'package:readr':

    col_factor
```

```
# 1. Fetch recent posts from Abdul El-Sayed's author feed
# (app.bsky.feed.getAuthorFeed accepts unauthenticated requests via public.api)
feed_req <- request("https://public.api.bsky.app/xrpc/app.bsky.feed.getAuthorFeed") |>
  req_url_query(
    actor  = "abdulelsayed.bsky.social",
    filter = "posts_no_replies", # Focus on primary posts
    limit  = 50
  )

feed_resp <- feed_req |> req_perform() |> resp_body_json()

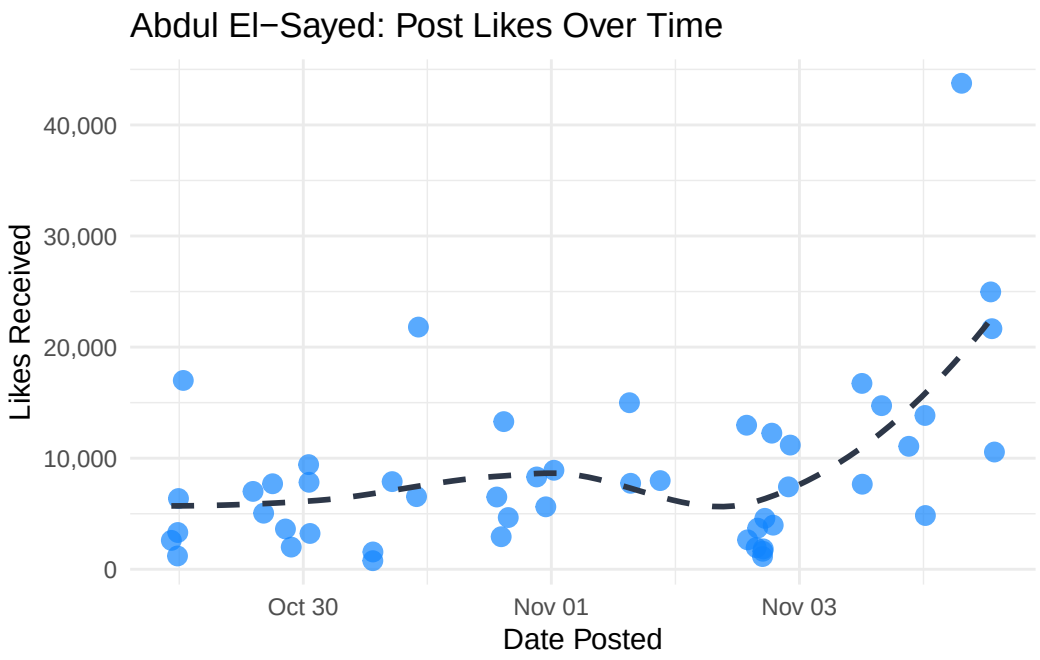
# 2. Extract post metrics into a clean dataframe
posts_df <- map_dfr(feed_resp$feed, function(item) {
  post <- item$post
  rec  <- post$record

  tibble(
    created_at = ymd_hms(rec$createdAt),
    likes      = post$likeCount %||% 0L,
    text       = rec$text %||% ""
  )
})

ggplot(posts_df, aes(x = created_at, y = likes)) +
  geom_point(aes(size = reposts), color = "#1185FE", alpha = 0.7, size = 3) +
  geom_smooth(method = "loess", se = FALSE, color = "#2D3748", linetype = "dashed") +
  scale_y_continuous(labels = comma) +
  labs(
    title = "Abdul El-Sayed: Post Likes Over Time",
    x = "Date Posted",
    y = "Likes Received",
    size = "Reposts"
  ) +
  theme_minimal()
```

```
Ignoring unknown labels:
* size : "Reposts"

`geom_smooth()` using formula = 'y ~ x'
```



20 Activity 2

Let's reflect on the type(s) of analysis needed for your social media data.

- Do you need to count things?
- Are you interested in comparative analysis?
- Do you want to perform text analysis on posts?
- Do you need to classify or perform sentiment analysis on the posts?
- Other possibilities?

21 Export results to CSV

You may (and should) write any new and derived data you want to keep to file. It's very easy to read it back into R to continue your work later.

```
# enter the name of the dataframe you wish to save and its file name

write_csv(academic_posts_df, "bluesky_academicsky_posts.csv")
```

22 A note on Gen AI

- I used Gemini to help develop the code examples in this workshop.
- I did not use AI to generate the text of this workshop. Writing is where I draw the line. At best, I use it as a copyeditor, when I'm done and need a grammar/spelling check.

23 Thanks

I'm happy to hear from users about how they end up applying what they see in workshops to questions of their own. To that end, feel free to be in touch with questions or feedback. A survey about this workshop will reach your inbox in an hour.